

ENTREGA PRUEBAS DE API Y MOCKING

CLASE 05

Pruebas sobre API, Mocking e Integración UI — Backend

EQUIPO HIGHMIND

Ignacio Ayala · Juan Manuel Humeniuk · Gonzalo Mascareño · Ignacio Morichetti

1. Tabla de Diseño de Pruebas

ID Caso	Tipo de Prueba	Objetivo	Entradas	Resultado Esperado
TC-01	API Real	Verificar que el endpoint de productos devuelve la lista activa correctamente.	Solicitud GET a /api/producto.php	Status 200 OK. La respuesta debe ser un arreglo JSON conteniendo objetos con atributos de producto (id, nombre, precio).
TC-02	API Real	Validar que no se puede agregar al carrito sin estar autenticado.	Solicitud POST a /api/carrito.php con { action: 'add', id: 1, qty: 1 } (sin cookie de sesión válida).	Status 401 Unauthorized. JSON de respuesta indicando {"ok": false, "msg": "No autorizado"}.
TC-03	Mocking	Comprobar que la UI maneja correctamente una caída del backend (Error 500) al listar productos.	Intercepción de red a **/api/producto* forzando un Status 500 y error simulado. Navegación a /frontend/index.html.	La UI no se rompe; se visualiza un mensaje amigable de error (.error-message) alertando al usuario.
TC-04	Mocking	Validar el "empty state" del catálogo de productos.	Intercepción de red a **/api/producto* retornando un array vacío []. Navegación a /frontend/index.html.	Se oculta la grilla de productos y aparece un mensaje de .no-products indicando que no hay productos disponibles.
TC-05	Híbrida	Validar el flujo de visualización de ítems en el carrito aislando la preparación de datos por API.	1. POST a /api/carrito.php agregando cantidad 3 del producto ID 1. 2. Navegación manual a /frontend/carrito.html.	La página del carrito carga y el input de cantidad para el producto reflejado en el DOM es exactamente '3'.

2. Estrategia Utilizada

Por qué API Real (TC-01, TC-02): El testing puro de API es fundamental para asegurar las reglas de negocio de Highmind Web en el backend (ej. autorización del carrito y formato de salida del catálogo). Es mucho más rápido que interactuar por UI y permite detectar fallas directamente en la capa de servicios.

Por qué Mocking (TC-03, TC-04): Replicar un error 500 real del servidor obligaría a bajar el servicio local, lo cual arruinaría las otras pruebas. El mocking a través de `page.route()` aísla este escenario complejo permitiendo verificar el comportamiento defensivo del frontend. De igual manera, simular un catálogo vacío evita tener que borrar (y luego restaurar) los productos de la base de datos real.

Por qué Prueba Híbrida (TC-05): Para validar que el carrito de compras visualice correctamente los items, si lo hiciéramos todo por UI tendríamos que buscar el producto, hacer clic, agregarlo, y luego ir al carrito (flujo muy largo y frágil “flaky”). Al preparar el estado (agregar al carrito) de forma directa por API y luego validar en UI, la prueba se vuelve muchísimo más rápida y enfocada estrictamente en probar la “renderización” del carrito.

3. Evidencia de Ejecución

Para ejecutar estas pruebas, se debe correr el comando `npx playwright test tests/clase-05/`. A continuación se muestra la salida exitosa de las pruebas:

```
PS C:\Users\gonza\Documents\GONZALO\Desar. de Soft ITU\3er Semestre\Metodologia y
Testing\HIGHMIND_WEB> npx playwright test tests/clase-05/

Running 5 tests using 1 worker

  ✓ 1 tests/clase-05/api.spec.js:6:5 › Pruebas de API - Highmind Web › Verificar obtención de
productos activos (345ms)
  ✓ 2 tests/clase-05/api.spec.js:21:5 › Pruebas de API - Highmind Web › Intentar agregar al
carrito sin estar autenticado (401) (122ms)
  ✓ 3 tests/clase-05/mockingspec.js:5:5 › Pruebas con Mocking/Intercepción - Highmind Web ›
Simular error 500 al intentar cargar productos y validar UI (610ms)
  ✓ 4 tests/clase-05/mockingspec.js:25:5 › Pruebas con Mocking/Intercepción - Highmind Web ›
Mockear lista de productos vacía para validar el "empty state" (480ms)
  ✓ 5 tests/clase-05/hybrid.spec.js:5:5 › Pruebas Híbridas - Highmind Web › Preparar carrito vía
API y validar visualización en UI (850ms)

5 passed (2.4s)
```